

1. ne comparăm

problema: inițializarea funcției $f()$ nu trebuie ascundea
restul funcțiilor de același nume din Base.

modificări: 15. d. Base :: $f(\Delta)$;

output: $\Delta 1$
 $B 2$

! $\text{int } f() \text{ const} \dots \rightarrow$ nu modifică variabile numere.

2. pointe + const

Sintaxa: 1. $\begin{cases} \text{int const } * p; \\ \text{const int } * p; \end{cases} \rightarrow$ pointer non-const la
val. const.

2. $\text{const int } * \text{const } p; \rightarrow$ pointer const la val
const

3. $\text{int } * \text{const } p; \rightarrow$ pointer const la val
non-const.

1. valoarea la care pointerul nu poate fi modificată pentru pointer.
adresa poate fi schimbată

3. valoarea la care pointerul poate fi modificată —
adresa nu poate fi schimbată.

2. nici valoarea, nici adresa nu pot fi schimbate după inițializare.

! Nu sunt modificări accidentale. $\rightarrow$ siguranța codului

Exemple: $\text{int } x = 1, y = 2;$

$\text{int const } * p = \&x;$ // nu se poate schimba $*p$,
se poate schimba p

$\text{int } * \text{const } z = \&y;$ // nu z , dar $*z$

$\text{const int } * \text{const } t = \&x;$ // nu t , nu $*t$

$*p++$; $\rightarrow$ NU

$p = \&y; \rightarrow$ DA

$*z++$; $\rightarrow$ DA

$z = \&y; \rightarrow$ NU

$*t++$; $\rightarrow$ NU

$t = \&y; \rightarrow$ NU

3. compilarea

output: 10 40 30 40 20 50 80 * 50 40 80 20
30 40 10

ordine constructor: → global / static global în ordine
→ main() în ordine

ordine destructor: → local non-static
→ local static
→ global / static global } ordine inversă

! atente la static + scope-ul funcțiilor

4. ordine constructor: → noțiune virtual: constructor la baza înaintea altor alți.
→ la rând: baza → membrii → derivata

ordine destructor: → derivate
→ membrii
→ non-virtual la baza
→ virtual la baza → la final ! } ordine inversă față de constructor

Derivata 5 → 1, 2, 3, 4
3 → virtual la baza

output: CB CB CA1 CB CA2 CA3 CB AD4 Derivata 5
↓ ↓ ↓ ↓
3 1 2 4
AD2 AD1

! atente la endline "\n"

! ceafica

noțiune

diminuata !

5. constructor protected / private

Sintaxa :

```
class Exemplu1 {  
    protected:  
    Exemplu1() { ... }  
};  
  
class Exemplu2 {  
    private:  
    Exemplu2() { ... }  
};
```

nu pot fi apelati direct din exteriorul clasei

Pot fi apelati doar → doar clase derivate (protected doar)
→ de metodele clasei respective
→ de clase friend

Particularitati: → nu se pot crea obiecte direct (Exemplu1 e1;
sau Exemplu2 e2; va fi eroare)
→ se pot crea obiecte doar in interiorul clasei
(private) sau in clasele derivate (protected).

→ Clase abstracte sau de baza (faptarea folosirii mostenirii
si pentru instantierea directa).
→ Singleton (pt. a controla numarul instantiilor)

6. ambiguitate

problema: ambiguitate cu functia afis(), se gaseste in
ansi multe locuri diferite, compilatorul nu stie exact
ce varianta de functie sa apeleze.

solutie: 18. ob. mii :: afis(); (poate fi ori mii :: ori ras ::,
in functie de a dori sa
afisam)

7. nu compilată

soluție: 15. (a+d).afis();

problema: d este tip Cls 2, a e tip Cls, nu există

o varianta a feretiei: Cls 2 operata +(Cls obj)...

=> eroare

8. Statica: class Baza {

public:

static void metodaStatica() { ... };

};

Proprietati generale:

- metodele statice apartin clasei, nu obiectelor
- pot fi apelate fara sa creste un obiect
- nu au acces la "this"
- pot accesa doar alti membrii statici ai clasei.

Particularitati la
nomenclatura:

- nu există ambiguitate la apelarea metodelor statice
- apelul se face numai specificând clasa
ex: f();
- dacă o clasă definește o metodă statică cu același nume, aceasta ascunde metodele statice cu același nume din clasele sale.

9. compilarea

output: Examen

! atentie la codurile speciale

10. un compilator

problema: pointer la valoare const, nu își poate modifica val. la conținut, dar poate schimba adresa.

soluție: $6. \text{ int } * \text{ const } a - \text{ ptr} = \&a;$

output: 6
 7

11. operatorul typeid.

Sintaxa: `typeid (expr)`

- permite identificarea tipului la compilare sau la rulare
- poate fi folosit pe tipuri (`typeid(int)`) sau expresii (`typeid(obj)`)
- obiecte polimorfe (al puțin o metodă virtuală) → returnează tipul dinamic (real, la rulare)
- obiecte non-polimorfe → returnează tipul static (la compilare).

Notiție: pentru identificarea tipului real al unui obiect la rulare, mai ales de erorile de clasă cu metode și funcții virtuale.

! destructoare virtuale dacă vom să ținem obiectul prin pointer la

bază: $\text{Baza } * p = \text{noua derivată};$
delete p;

12. compilator

output: Baza

! throw obiect → tipul dinamic

throw referință / pointer → tipul static

• polimorfe → ca mai sus

13. nu compilată

problema: ambele ob. d. obz. d. printr-o dată acuză
zona în memorie. ~ Δ(1) face date d; pt.
ambele obiecte ⇒ acuză zona în memorie e
ștearsă de 2 ori.

soluție: nu există soluție cu să modifice doar o linie și
să modifice id-ul original a codului.

Comentarii / sugere linia 13.

14. Sintaxa:

```
class Baza {  
    public:  
        virtual ~Baza() = 0;  
};
```

inline Baza::~Baza(){}; // masura
implementarea

→ Juc clase abstracte (nu se pot crea obiecte de acel tip).
→ Toate clasele derivate trebuie să își implementeze destructor
sau să fie abstracte.
→ permite sugere creșterea a obiectelor derivate prin parinti
la baza.

Notificație: ierarhia de clase polymorfe, când vrem să portăm
abstractizarea clasei de bază și să asigurăm distrugerea
corectă a obiectelor derivate.

15. compilată

output: 2

16. ne compilatează

problema: referința la linia 5.

soluție: `int get() const { return c; }`

sau

`const int & get() const { ... }`

17. Sintaxa: `class A {
public:
 A(const A& obj) {
 ...
 }
};`

- permite o referință constantă la un obiect de acelasi tip
- apelat automat la:
 - inițializare cu alt obiect
 - transfer / returnare prin val. a obiectelor
- necesar când clasa gestionează resurse din memorie, pt a evita shallow copy, double free, memory leak.
- recomandată definirea operatorilor de atribuire și destructor (Regula celor 3).
- folosit pt. deep copy (copierea resurselor, nu doar a adresei).

motivație: pt. a controla modul în care se copiază obiectele

18. compilată

output: instanțieri c: 23 val elem: 22

! atenție la scope

Nr 2

1. nu compilată

problema: pointerul a_ptr este un pointer non-const la
valoare const, dar a nu e permis.

soluție: facem a_ptr pointer const la val. non-const,
pentru a putea schimba val. lui a pe la autumntul lui
a_ptr.

output: 8
9

2. compilată

output: 1 3 4 5 2 6 7 * 6 5 7 2 4 3 1

3. nu compilată

problema: nu există o funcție cu 2 operatori + (cls ob);
⇒ (d+a).afis(); do euare

soluție: (a+d).afis(); → cls operator + (cls ob);

4. compilată

output: Bota

5. nu compilată

problema: int & cannot bind const int

soluție: ștergem referința sau adăugăm const la input.

6. compilată

output: 2025
↳ 700

7. compilată

output : instanțeri C : 13 val elem : 11

8. nu compilată

problema : apel ambiguu la funcția afis().

soluție : ob. oras :: afis(); sau ob. uni :: afis();

Specificăm explicit cu funcție reu să fie apelată.

9. compilată

output : 6

10. declarația lui f() nu duce la ascunderea funcției din
bază

soluție : d. Base :: f();